Distributed System

MapReduce

Lab1

This Lab is a high-performance implementation of the Google MapReduce framework. It is designed to handle large-scale data processing by distributing computation across a cluster of workers managed by a central coordinator.

CONTENTS

01

introduction

introduction

In this lab you'll build a MapReduce system. You'll implement a worker process that calls application Map and Reduce functions and handles reading and writing files, and a coordinator process that hands out tasks to workers and copes with failed workers.

MapReduce

MapReduce

what is MapReduce ?

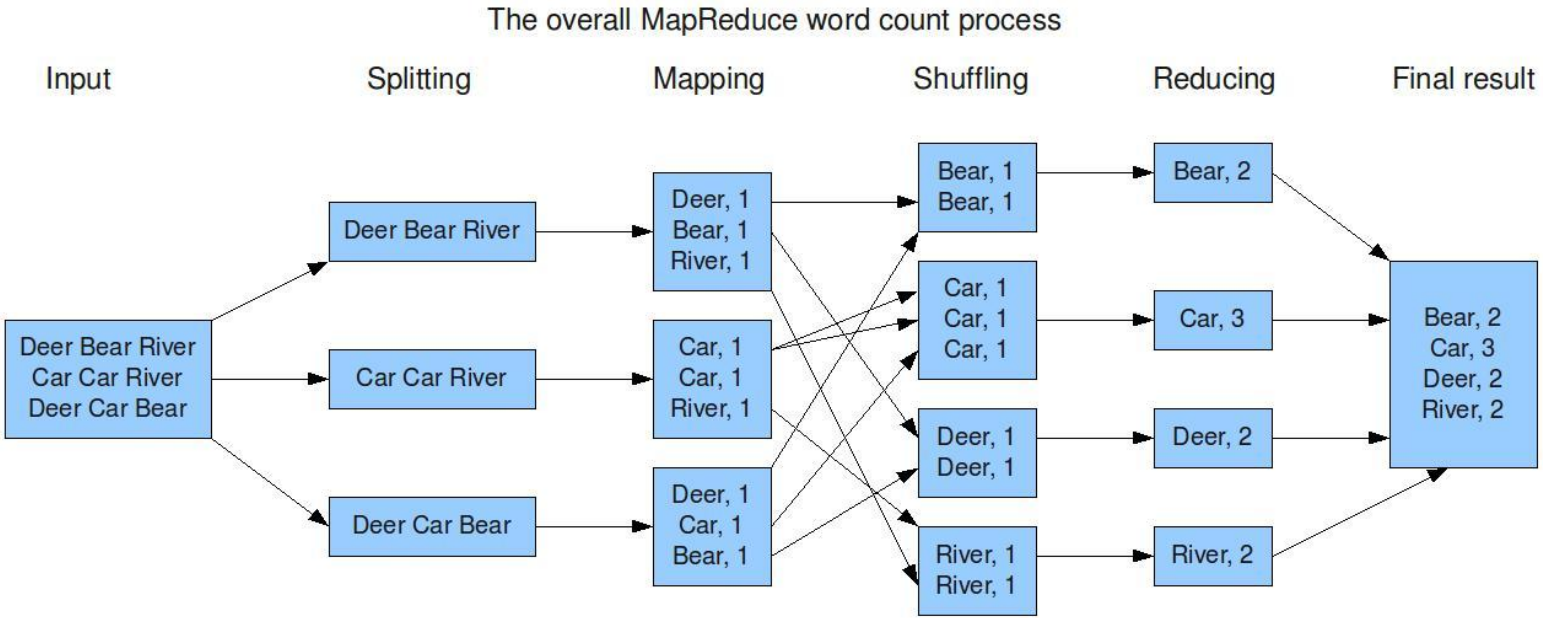
MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a map function that processes a key/value pair to generate a set of intermediate key/value pairs, and a reduce function that merges all intermediate values associated with the same intermediate key.

1. The Map Phase Input: A large file is split into smaller "chunks."Process: The worker reads a chunk and applies the user-defined Map function.Output: A series of intermediate Key/Value pairs (e.g., {"apple": 1, "orange": 1}).

2. The Shuffle & Sort Phase Partitioning: The system uses a hash function—**hash(key)** mod **Nreduce**—to ensure that all instances of the same key end up on the same reducer. The data is sorted by key so that identical keys are grouped together, making the final aggregation simple.

3. The Reduce PhaseProcess: The worker takes a unique key and the entire list of values associated with it (e.g., {"apple": [1, 1, 1, 1]}).**Output:** It "reduces" that list into a single result (e.g., {"apple": 4}). **Final Result:** All reducer outputs are combined into a final set of files.

MapReduce


The overall MapReduce word count process
Input
Splitting
Mapping
Shuffling
Reducing
Final result
Deer Bear River
Car Car River
Deer Car Bear
Deer Bear River
Car Car River
Deer Car Bear
Deer, 1
Bear, 1
River, 1
Car, 1
Car, 1
River, 1
Deer, 1
Car, 1
Bear, 1
Bear, 1
Bear, 1
Car, 1
Car, 1
Car, 1
Deer, 1
Deer, 1
River, 1
River, 1
Bear, 2
Car, 3
Deer, 2
River, 2
Bear, 2
Car, 3
Deer, 2
River, 2

MapReduce in distributed system

MapReduce in distributed system

MapReduce: A Distributed Systems PerspectiveIn a distributed environment, the goal is to solve the "Three V's" (Volume, Velocity, Variety) by spreading the load across a cluster of machines. MapReduce is the framework that handles the Coordination, Communication, and Consistency required to make this happen.

1. Distributed Coordination (The Master/Worker Pattern)In a decentralized environment, you need a "Source of Truth."The Coordinator (**Centralized Control**): Acts as the scheduler. It manages the global state and assigns tasks based on data locality (assigning work to the node where the data already lives to save bandwidth).Worker Nodes (The Fleet): Independent processes that handle the actual compute. They are "**stateless**," meaning if one dies, the system doesn't lose its mind; it simply reassigns the task.

2. RPC: The Communication Fabric Since workers and coordinators live on different machines (or processes), they cannot share memory. **Remote Procedure Calls (RPC):** Used to bridge the gap. When a worker says AskTask(), it is performing a network call that looks like a local function but actually crosses the network.The Shared Storage Layer: Instead of sending massive data chunks through the coordinator (which would create a bottleneck), workers write to a **Distributed File System (DFS)**. Mappers write, and Reducers read from specific "buckets."

3. **Fault Tolerance** (Resilience in Chaos)In a distributed system, hardware failure is a statistical certainty, not an accident.Detecting Failure: The coordinator uses a "Heartbeat" mechanism. If a worker doesn't report back within 10 seconds, it is marked as dead.**Re-execution:** Because the Map and Reduce functions are idempotent (running them twice produces the same result), the coordinator can safely restart a failed task on a new node without corrupting the final output.

4. The "Shuffle": Distributed Data MovementThe most expensive part of a distributed system is the network.Data Partitioning: MapReduce minimizes "all-to-all" communication. Workers use **hash(key)**mod **Nreduce** to ensure that data only moves to the specific Reducer that needs it. **Grouping**: By sorting data on the worker side before sending it, the system reduces the CPU load on the receiving Reducers.

5. Consistency & Atomic Commits **How do you prevent a slow worker from overwriting the work of a fast worker?** **Atomic Renames:** Workers write results to temporary files. Once finished, they perform an atomic os.Rename. In a distributed file system, this acts as a **Commit**, ensuring that the final output mr-out-X is never in a "**half-written**" state.

Coordinator

```
type TaskState int

const (
    TaskStateIdle TaskState = iota
    TaskStateInProgress
    TaskStateCompleted
)
```

Represents the state of a task:

Idle → not started
InProgress → currently running
Completed → finished

Represents the current phase:

PhaseMap → map tasks running
PhaseReduce → reduce tasks running
PhaseDone → all work finished

```
type CoordinatorPhase int

const (
    PhaseMap CoordinatorPhase = iota
    PhaseReduce
    PhaseDone
)
```

Coordinator

```
type Task struct {
    State     TaskState
    StartTime time.Time
    Filename  string
}
```

Each task contains:

State → current status
StartTime → when execution started (used for timeout)
Filename → only used in Map phase

```
type Coordinator struct {
    mu          sync.Mutex
    Phase       CoordinatorPhase
    MapTasks    []Task
    ReduceTasks []Task
    NReduce     int
    NMap        int
}
```

mu → mutex for sync safety
Phase → current stage
MapTasks → list of map tasks
ReduceTasks → list of reduce tasks

Coordinator

Step 1: Worker Requests a Task

A worker sends an RPC request by calling AskTask.
This means the worker is idle and ready to do some work.

Step 2: Coordinator Locks Shared Data

The Coordinator uses a mutex (c.mu.Lock()) to ensure that only one worker at a time can access and modify the task list.
This prevents two workers from receiving the same task.

Step 3: Coordinator Checks Current Phase

The Coordinator looks at its current phase:

If the phase is Map, it will assign map tasks
If the phase is Reduce, it will assign reduce tasks
If the phase is Done, it will tell the worker to exit

Step 4: Assigning a Map Task (Map Phase)

If the system is in the Map phase, the Coordinator scans through all map tasks.

It looks for a task whose state is Idle (not yet assigned)
Once it finds one:
It marks the task as InProgress
It records the current time as the task's start time
It sends the task details back to the worker

The worker receives:

Task type (Map)
Task ID
Input filename
Number of reduce tasks
Total number of map tasks

This allows the worker to start processing the file.

Step 5: No Available Map Task

If all map tasks are already assigned (none are Idle), but some are still running:

The Coordinator tells the worker to Wait
The worker will sleep for a short time and then ask again later

Step 6: Assigning a Reduce Task (Reduce Phase)
After all map tasks are completed, the Coordinator switches to the Reduce phase.

Now, when a worker asks for a task:
The Coordinator scans reduce tasks
Finds an Idle reduce task
Marks it as InProgress
Sends the task ID and related information to the worker
Note: Reduce tasks do not include a filename because they work on intermediate data.

Step 7: No Available Reduce Task
If all reduce tasks are assigned but not finished yet:
The Coordinator tells the worker to Wait
The worker will retry later

```go
func (c *Coordinator) AskTask(args *AskTaskArgs, reply *AskTaskReply) error {
    c.mu.Lock()
    defer c.mu.Unlock()

    if c.Phase == PhaseMap {
        for i := range c.MapTasks {
            if c.MapTasks[i].State == TaskStateIdle {
                c.MapTasks[i].State = TaskStateInProgress
                c.MapTasks[i].StartTime = time.Now()
                reply.TaskType = TaskTypeMap
                reply.TaskID = i
                reply.Filename = c.MapTasks[i].Filename
                reply.NReduce = c.NReduce
                reply.NMap = c.NMap
                return nil
            }
        }
        // If all map tasks are assigned but not completed, wait
        reply.TaskType = TaskTypeWait
        return nil
    } else if c.Phase == PhaseReduce {
        for i := range c.ReduceTasks {
            if c.ReduceTasks[i].State == TaskStateIdle {
                c.ReduceTasks[i].State = TaskStateInProgress
                c.ReduceTasks[i].StartTime = time.Now()
                reply.TaskType = TaskTypeReduce
                reply.TaskID = i
                reply.NReduce = c.NReduce
                reply.NMap = c.NMap
                return nil
            }
        }
        reply.TaskType = TaskTypeWait
        return nil
    } else {
        reply.TaskType = TaskTypeExit
        return nil
    }
}
```

Step 8: All Work Completed

If both map and reduce tasks are finished:

The Coordinator sets the phase to Done
Any worker that asks for a task will receive an Exit signal
The worker then terminates

Step 9: Task Reassignment (Failure Handling)

If a worker takes a task but does not finish it within a certain time (e.g., 10 seconds):

The Coordinator resets the task state back to Idle
This allows another worker to pick it up

This ensures the system continues even if a worker crashes or becomes slow.

```go
func (c *Coordinator) ReportTask(args *ReportTaskArgs, reply *ReportTaskReply) error {
    c.mu.Lock()
    defer c.mu.Unlock()

    if args.TaskType == TaskTypeMap && c.Phase == PhaseMap {
        if c.MapTasks[args.TaskID].State == TaskStateInProgress {
            c.MapTasks[args.TaskID].State = TaskStateCompleted
            // Check if all map tasks are done
            allDone := true
            for i := range c.MapTasks {
                if c.MapTasks[i].State != TaskStateCompleted {
                    allDone = false
                    break
                }
            }
            if allDone {
                c.Phase = PhaseReduce
            }
        }
    } else if args.TaskType == TaskTypeReduce && c.Phase == PhaseReduce {
        if c.ReduceTasks[args.TaskID].State == TaskStateInProgress {
            c.ReduceTasks[args.TaskID].State = TaskStateCompleted
            // Check if all reduce tasks are done
            allDone := true
            for i := range c.ReduceTasks {
                if c.ReduceTasks[i].State != TaskStateCompleted {
                    allDone = false
                    break
                }
            }
            if allDone {
                c.Phase = PhaseDone
            }
        }
    }
    return nil
}
```

```go
func (c *Coordinator) checkTimeouts() {
    for {
        time.Sleep(2 * time.Second)
        c.mu.Lock()
        if c.Phase == PhaseMap {
            for i := range c.MapTasks {
                if c.MapTasks[i].State == TaskStateInProgress && time.Since(c.MapTasks[i].StartTime) > 10*time.Second {
                    c.MapTasks[i].State = TaskStateIdle
                }
            }
        } else if c.Phase == PhaseReduce {
            for i := range c.ReduceTasks {
                if c.ReduceTasks[i].State == TaskStateInProgress && time.Since(c.ReduceTasks[i].StartTime) > 10*time.Second {
                    c.ReduceTasks[i].State = TaskStateIdle
                }
            }
        } else {
            c.mu.Unlock()
            break
        }
        c.mu.Unlock()
    }
}
```

Coordinator

1. func (c *Coordinator) server(sockname string)

This function sets up the RPC (Remote Procedure Call) server, allowing Workers to talk to the Coordinator.

rpc.Register(c): This publishes the Coordinator object's methods (like AskTask and ReportTask) so they can be called remotely by Workers.

os.Remove(sockname): In Unix systems, socket files persist even after a program crashes. This line ensures any "old" socket is deleted so a fresh one can be created.

net.Listen("unix", sockname): Tells the operating system to listen for incoming connections on a Unix Domain Socket (faster than TCP for communication on the same machine).

go http.Serve(l, nil): Starts a new Goroutine to handle incoming requests. This is non-blocking, meaning the Coordinator can continue running its own logic while the server waits for Workers in the background.

2. func (c *Coordinator) Done() bool

This function acts as the "Shutdown Switch" for the entire system.

Periodic Polling: The main program (the one that started the Coordinator) calls this function every second.

Thread Safety: It uses c.mu.Lock() to safely check the current phase.

The Logic: It returns true only if c.Phase == PhaseDone. Once it returns true, the main process knows all work is finished and it is safe to close the program.

3. func MakeCoordinator(...) *Coordinator

This is the Constructor (the entry point). It initializes the entire distributed system.

Phase Initialization: It starts the system in PhaseMap.

Task Creation:

Map Tasks: It creates one Task for every input file provided in the files slice.

Reduce Tasks: It creates nReduce number of tasks (specified by the user).

Setting the State: All tasks are initially marked as TaskStateIdle.

Activating Background Processes:

It calls c.server() to start listening for Workers.

It calls go c.checkTimeouts() to start the monitoring for crashed workers.

Return: It returns a pointer to the fully configured Coordinator object.

Coordinator

```go
// start a thread that listens for RPCs from worker.go
func (c *Coordinator) server(sockname string) {
    rpc.Register(c)
    rpc.HandleHTTP()
    os.Remove(sockname)
    l, e := net.Listen("unix", sockname)
    if e != nil {
        log.Fatalf("listen error %s: %v", sockname, e)
    }
    go http.Serve(l, nil)
}

// main/mrcoordinator.go calls Done() periodically to find out
// if the entire job has finished.
func (c *Coordinator) Done() bool {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.Phase == PhaseDone
}

// create a Coordinator.
```

```go
// nReduce is the number of reduce tasks to use.
func MakeCoordinator(sockname string, files []string, nReduce int) *Coordinator {
    c := Coordinator{}

    c.Phase = PhaseMap
    c.NReduce = nReduce
    c.NMap = len(files)

    c.MapTasks = make([]Task, len(files))
    for i, f := range files {
        c.MapTasks[i] = Task{
            State:    TaskStateIdle,
            Filename: f,
        }
    }

    c.ReduceTasks = make([]Task, nReduce)
    for i := 0; i < nReduce; i++ {
        c.ReduceTasks[i] = Task{
            State: TaskStateIdle,
        }
    }

    c.server(sockname)

    go c.checkTimeouts()

    return &c
}
```

RPC

```go
type TaskType int

const (
    TaskTypeMap TaskType = iota
    TaskTypeReduce
    TaskTypeWait
    TaskTypeExit
)

type AskTaskArgs struct {
}

type AskTaskReply struct {
    TaskType TaskType
    TaskID   int
    Filename string
    NReduce  int
    NMap     int
}

type ReportTaskArgs struct {
    TaskType TaskType
    TaskID   int
}

type ReportTaskReply struct {
}
```

TaskTypeMap
Worker should execute a Map task
Process an input file
TaskTypeReduce
Worker should execute a Reduce task
Aggregate intermediate results
TaskTypeWait
No task available now
Worker should wait and retry later
TaskTypeExit
All work is finished
Worker should terminate
AskTaskArgs RPC
Empty struct
Worker doesn't need to send anything
Just says: "I'm ready for work"
ReportTask RPC
After finishing work, the worker calls this.
Worker sends:
TaskType → Map or Reduce
TaskID → which task was completed
Coordinator doesn't need to send anything back

Worker

```go
// Map functions return a slice of KeyValue.
type KeyValue struct {
    Key   string
    Value string
}

// use ihash(key) % NReduce to choose the re
// task number for each KeyValue emitted by
func ihash(key string) int {
    h := fnv.New32a()
    h.Write([]byte(key))
    return int(h.Sum32() & 0x7fffffff)
}
```

1: KeyValue Data Structure
This is the basic data format:

Key → word or identifier
Value → usually a count or string

Example:

("apple", "1")

2:Hash Function
This function:
Converts a key into a number
Used to decide which Reduce task gets the data

Formula:
hash(key) % NReduce
This ensures:
Same key always goes to same reducer

Worker

1. The Infinite Work Loop

The for { ... } block ensures the worker stays alive and keeps processing tasks until the entire job is finished. A worker doesn't just do one task and quit; it continuously asks for more.

2. Communicating with the Coordinator

AskTask RPC: The worker starts by calling Coordinator.AskTask. It’s essentially saying, "I’m idle, what should I do?"

Handling the Response (reply.TaskType):

TaskTypeExit: If the job is complete, the worker breaks the loop and shuts down gracefully.

TaskTypeWait: If all tasks are currently assigned but the phase isn't finished (e.g., waiting for a slow mapper to finish before starting the reduce phase), the worker sleeps for 1 second. This prevents "busy waiting" (spamming the CPU).

3. Map Task Execution (The Logic)

If the Coordinator assigns a Map Task, the worker follows these steps:

File Input/Output (I/O):

It takes the reply.Filename provided by the Coordinator.

It opens the file and reads the entire content into memory (ioutil.ReadAll).

Executing the User's Logic:

It calls mapf(filename, content).

Note: mapf is a "plugin" function. The worker doesn't know what it's counting (could be words, URLs, or prime numbers); it just passes the data to the function and gets back a slice of KeyValue pairs (kva).

```go
// main/mrworker.go calls this function.
func Worker(sockname string, mapf func(string, string) []KeyValue,
    reducef func(string, []string) string) {

    coordSockName = sockname

    for {
        args := AskTaskArgs{}
        reply := AskTaskReply{}

        ok := call("Coordinator.AskTask", &args, &reply)
        if !ok || reply.TaskType == TaskTypeExit {
            break
        }

        if reply.TaskType == TaskTypeWait {
            time.Sleep(time.Second)
            continue
        }

        if reply.TaskType == TaskTypeMap {
            // Read the file
            file, err := os.Open(reply.Filename)
            if err != nil {
                log.Fatalf("cannot open %v: %v", reply.Filename, err)
            }
            content, err := ioutil.ReadAll(file)
            if err != nil {
                log.Fatalf("cannot read %v: %v", reply.Filename, err)
            }
            file.Close()

            kva := mapf(reply.Filename, string(content))

            // create intermediate files
            outFiles := make([]*os.File, reply.NReduce)
            encoders := make([]*json.Encoder, reply.NReduce)
            for i := 0; i < reply.NReduce; i++ {
                tempOutput, _ := ioutil.TempFile("", fmt.Sprintf("mr-%v-%v-tmp-*", reply.TaskID, i))
                outFiles[i] = tempOutput
                encoders[i] = json.NewEncoder(tempOutput)
            }
```

worker

1. Establishing the Connection (rpc.DialHTTP)

The "Dial": Just like a phone call, the Worker needs to connect to the Coordinator's address.
Unix Domain Sockets: The code uses "unix" and coordSockName.

Distributed Insight: While distributed systems usually use TCP/IP to talk across different computers, this implementation uses Unix Sockets for high-performance communication between processes on the same machine. It avoids the overhead of the network stack.

Error Handling: If the Coordinator isn't running, DialHTTP will fail, and the Worker will terminate with a log.Fatal.

2. Resource Management (defer c.Close())

This is a Go best practice. It ensures that no matter how the function ends (whether the call succeeds or fails), the network connection is closed. This prevents socket leakage, which could eventually crash the operating system by running out of file descriptors.

3. The Remote Execution (c.Call)

This is where the "magic" of RPC happens.

Arguments: * rpcname: The string name of the function to run on the Coordinator (e.g., "Coordinator.AskTask").

args: The data the Worker is sending.
reply: A pointer where the Coordinator's response will be stored.

Abstraction: To the Worker, this feels like calling a local function. Under the hood, the net/rpc library is serializing the data, sending it over the socket, waiting for the Coordinator to process it, and bringing the result back.

4. Error Detection & Logging
Success: Returns true if the Coordinator processed the request successfully.

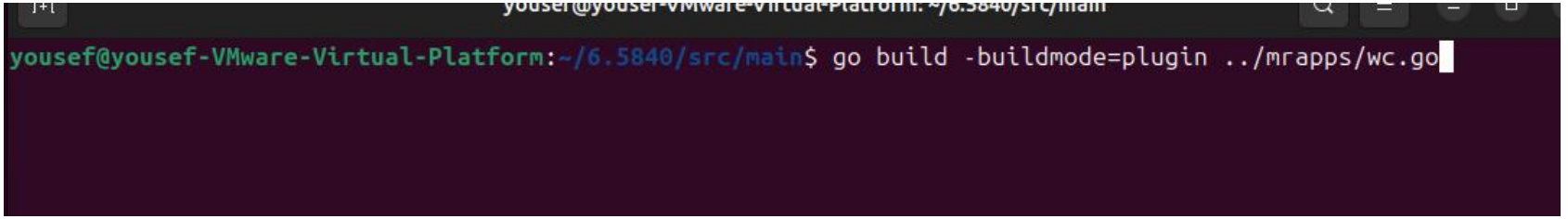
Failure: If the network is interrupted or the Coordinator method returns an error, it logs the Process ID (os.Getpid()) and the error message, then returns false.

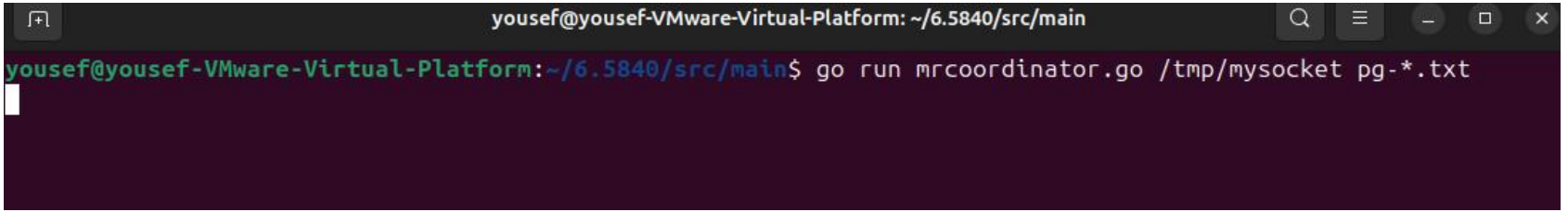
Why PID? Since you might have 10 workers running at once, logging the PID helps you identify which specific worker is having trouble

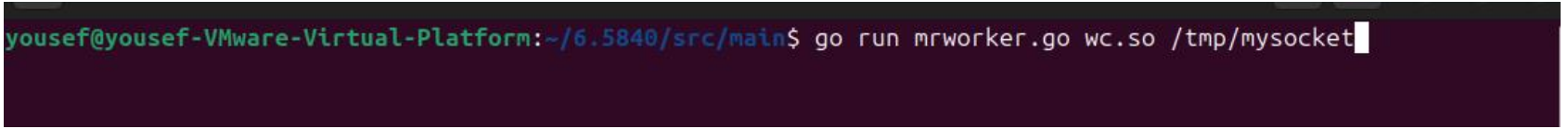
```go
// returns false if something goes wrong.
func call(rpcname string, args interface{}, reply interface{}) bool {
    // c, err := rpc.DialHTTP("tcp", "127.0.0.1"+":1234")
    c, err := rpc.DialHTTP("unix", coordSockName)
    if err != nil {
        log.Fatal("dialing:", err)
    }
    defer c.Close()

    if err := c.Call(rpcname, args, reply); err == nil {
        return true
    }
    log.Printf("%d: call failed err %v", os.Getpid(), err)
    return false
}
```

04

Simulation


yousef@yousef-VMware-Virtual-Platform:~/6.5840/src/main$ go build -buildmode=plugin ../mrapps/wc.go


yousef@yousef-VMware-Virtual-Platform: ~/6.5840/src/main
yousef@yousef-VMware-Virtual-Platform:~/6.5840/src/main$ go run mrcoordinator.go /tmp/mysocket pg-*.txt


yousef@yousef-VMware-Virtual-Platform:~/6.5840/src/main$ go run mrworker.go wc.so /tmp/mysocket